# Design & Analysis of Algorithm
# Lecture 1



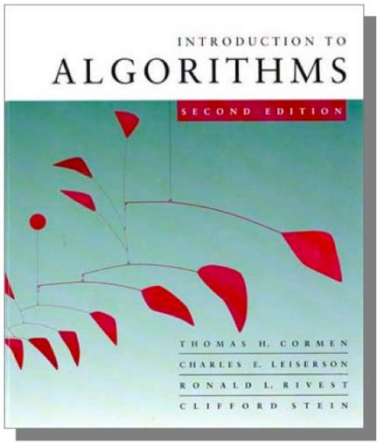
Department of Computer & information Science

# Materials

## ❖Textbook

- ***Introduction to Algorithms 2nd*** ,*Cormen, Leiserson, Rivest and Stein*, The MIT Press, 2001.
- ***Data Structures and Algorithm Analysis***

*Clifford A. Shaffer,Edition 3.2 (C++ Version),2012.*




## ❖Others

- ***Introduction to Design & Analysis Computer Algorithm 3rd,*** Sara Baase, Allen Van Gelder, Adison-Wesley, 2000.
- ***Algorithms***, Richard Johnsonbaugh, Marcus Schaefer, Prentice Hall, 2004.
- ***Introduction to The Design and Analysis of Algorithms 2nd Edition,*** Anany Levitin, Adison-Wesley, 2007.

# Course Objectives

- This course introduces students to the analysis and design of computer algorithms. Upon completion of this course, students will be able to do the following:
  - Analyze the asymptotic performance of algorithms.
  - Demonstrate a familiarity with major algorithms and data structures.
  - Analyze and Design Algorithms
  - Compare the performance of Algorithms..

# Objective of course

In this course we will cover the following topics:

- Understand foundations of algorithms && design and analysis various variants algorithms

  Accuracy

  Efficiency

  Comparing efficiencies

- Make use of skills to understand mathematical notations in algorithms and their simple mathematical proofs

- Gain familiarity with a number of classical problems that occur frequently in real-world applications

Thursday, September 04, 2025

# Advanced Algo Analysis

## COURSE OUTLINE

- **Preliminaries** : different types of algorithms, analyzing methodologies, notations, proof techniques and limits.
- **Asymptotic Notation** : different notations and their examples, standard notations and their common functions.
- **Analysis of Algorithms** : analyzing control structures, using barometer instruction, amortization, and different examples for analysis and solving recurrences.
- **Structures**: use of arrays, stacks, queues, records, pointers, lists, graphs, trees, hash tables, heaps and binomial heaps.
- **Searching/Sorting Algorithms** : Various searching and sorting algorithms and their comparisons.

## COURSE OUTLINE

- Tree Algorithms: understanding and making different types of trees and their algorithms.
- Graph Algorithms: understanding and making different types of graphs and their algorithms for finding shortest paths
- Heap Algorithms : developing heaps, binomial heaps, disjoint structures, and their algorithms.
- Divide and Conquer: Multiplication of Integers, Binary Search, Sorting Algorithms and Matrix Multiplication.
- Greedy Algorithms : General Characteristics of Greedy Algorithms, Minimum Spanning Trees, Shortest Graph Paths, Huffman Codes and Scheduling.
- NP-Completeness: understanding of P, NP and NP-completeness and use of algorithms for some problems

# What is Algorithm?

- Algorithm
  - is any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.
  - is thus a sequence of computational steps that transform the input into the output.
  - is a tool for solving a well - specified computational problem.
  - Any special method of solving a certain kind of problem (Webster Dictionary)

# ALGORITHMICS

It is the science that lets designers study and evaluate the effect of algorithms based on various factors so that the best algorithm is selected to meet a particular task in given circumstances. It is also the science that tells how to design a new algorithm for a particular job.

# What is a program?

- A program is the expression of an algorithm in a programming language
- a set of instructions which the computer will follow to solve a problem



# Introduction

- Why need algorithm analysis ?
  - writing a working program is not good enough
  - The program may be inefficient!
  - If the program is run on a large data set, then the running time becomes an issue



# Types of Algorithms

# PROBABILISITIC ALGORITHM

- In this algorithm, chosen values are used in such a way that the probability of chosen each value is known and controlled.

e.g. Randomize Quick Sort

# HEURISTIC ALGORITHM

- **This type of algorithm is based largely on optimism and often with minimal theoretical support. Here error can not be controlled but may be estimated how large it is.**

# APPROXIMATE ALGORITHM

In this algorithm, answer is obtained that is as précised as required in decimal notation. In other words it specifies the error we are willing to accept.

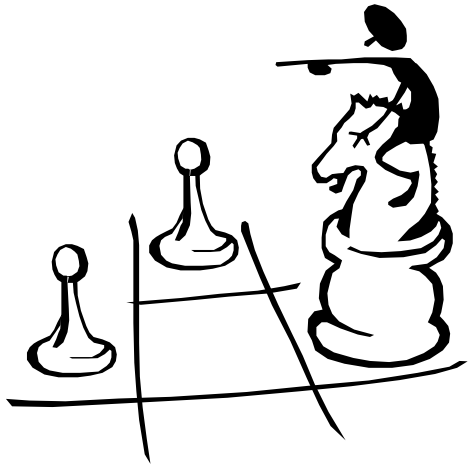
For example, two figures accuracy or 8 figures or whatever is required.

# Where We're Going

- Learn general approaches to algorithm design
  - Divide and conquer
  - Greedy method
  - Dynamic Programming
  - Basic Search and Traversal Technique
  - Graph Theory
  - Linear Programming
  - Approximation Algorithm
  - NP Problem

# Some Application

- Study problems these techniques can be applied to
  - sorting
  - data retrieval
  - network routing
  - Games
  - etc



# What do we analyze about them?

- Correctness
  - Does the input/output relation match algorithm requirement?
- Amount of work done (aka complexity)
  - Basic operations to do task finite amount of time
- Amount of space used
  - Memory used

# Strengthening the Informal Definition

- Important Features:
  - Finiteness.[algo should end in finite amount of steps]
  - Definiteness.[each instruction should be clear]
  - Input.[valid input clearly specified ]
  - Output.[single/multiple valid output]
  - Effectiveness.[ steps are sufficiently simple and basic]

# RAM model

- has one processor
- executes one instruction at a time
- each instruction takes "unit time“
- has fixed-size operands, and
- has fixed size storage (RAM and disk).



# Example Of Algorithm



# Few Classical Examples

Classical Multiplication Algorithms

- English
- American
- A la russe
- Divide and Conquer

# Classic Multiplication

**CLASSIC MULTIPLICATION ALGORITHMS**

**(981 x 1234)**

| American | English |
|---|---|
| 981 | 981 |
| 1234 | 1234 |
| 3924 | 981 |
| 2943 | 1962 |
| 1962 | 2943 |
| 981 | 3924 |
| 1210554 | 1210554 |

# Multiplication

## MULTIPLICATION (981 x 1234)

### (*a la russe algorithm*)

| | | |
|---|---|---|
| 981 | 1234 | 1234 |
| 490 | 2468 | |
| 245 | 4936 | 4936 |
| 122 | 9872 | |
| 61 | 19744 | 19744 |
| 30 | 39488 | |
| 15 | 78976 | 78976 |
| 7 | 157952 | 157952 |
| 3 | 315904 | 315904 |
| 1 | 631808 | 631808 |
| | | 1210554 |

# Multiplication

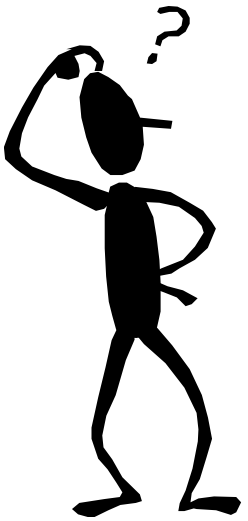
## MULTIPLICATION (981 x 1234)

***(Divide-and-Conquer Algorithm)***

| | Multiply | | Shift | Result |
|---|---|---|---|---|
| i) | 09 | 12 | 4 | 108…. |
| ii) | 09 | 34 | 2 | 306.. |
| iii) | 81 | 12 | 2 | 972.. |
| iv) | 81 | 34 | 0 | 2754 |
| | | | | **1210554** |

# The Selection Problem

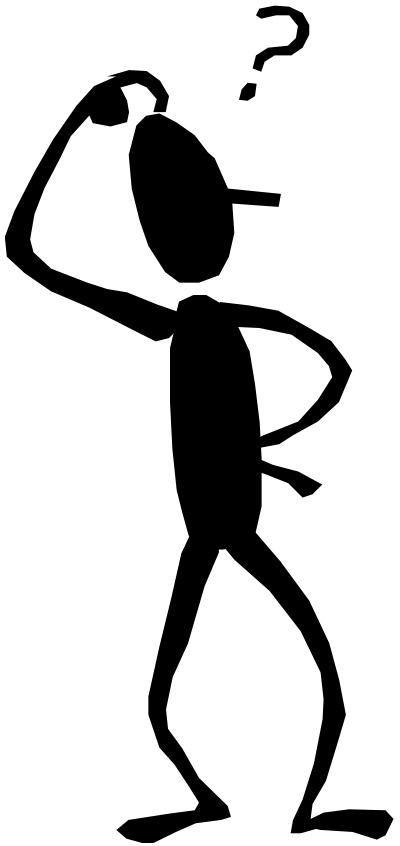
Which algorithm is better?



# Which algorithm is better?

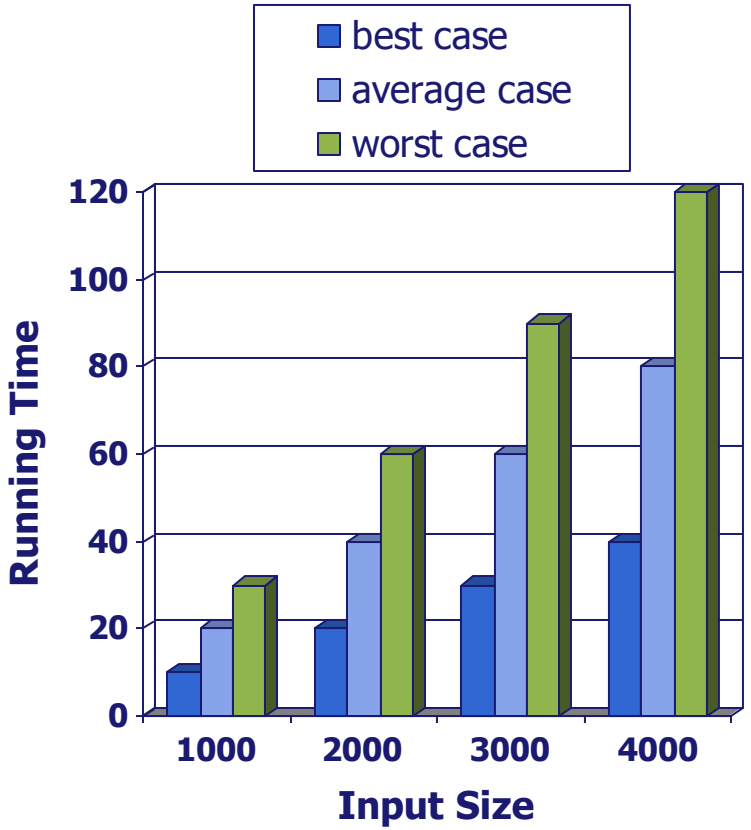
**The algorithms are correct, but which is the best?**

- Measure the running time (number of operations needed).
- Measure the amount of memory used.
- Note that the running time of the algorithms increase as the size of the input increases.



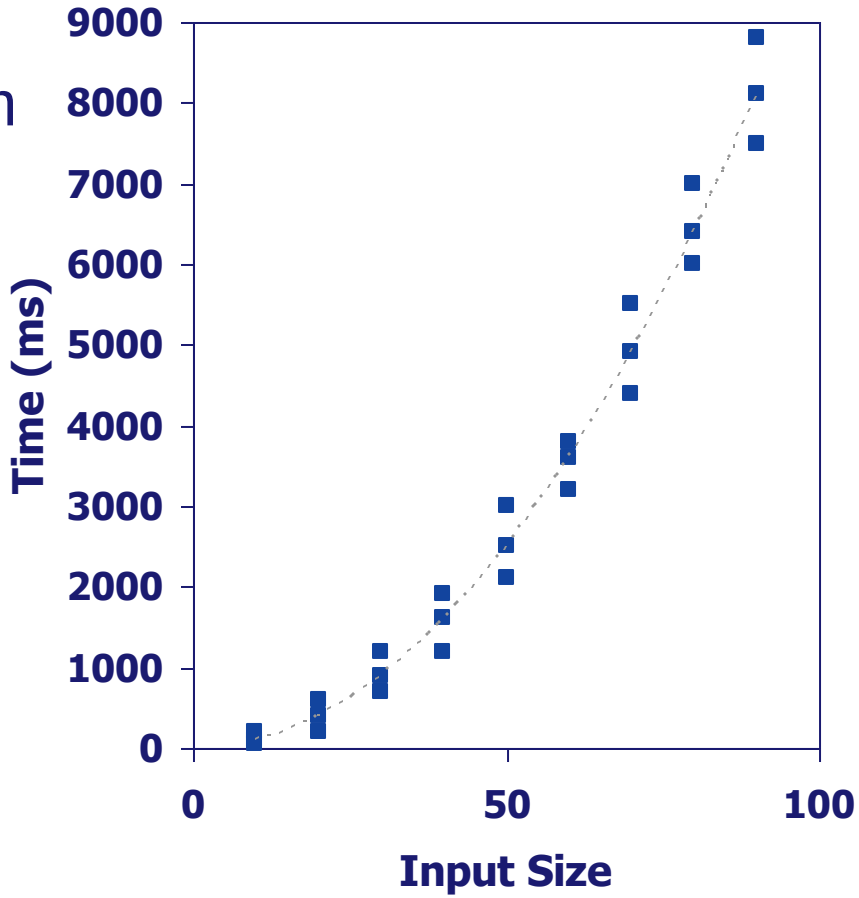
# Running Time

- Most algorithms transform input objects into output objects.
- The running time of an algorithm typically grows with the input size.
- Average case time is often difficult to determine.
- We focus on the worst case running time.
  - Easier to analyze
  - Crucial to applications such as games, finance and robotics



# Experimental Studies

- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition
- Use a method like System.currentTimeMillis() to get an accurate measure of the actual running time
- Plot the results



# Limitations of Experiments

- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments must be used

# Theoretical Analysis



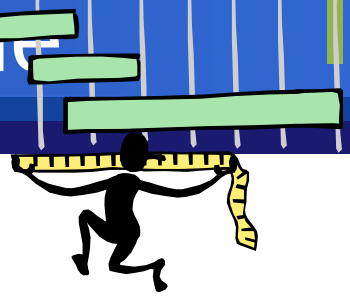
- Uses a high-level description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, *n*.
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

# Counting Primitive Operations

- By inspecting the pseudo code, we can determine the maximum number of primitive/basic operations executed by an algorithm, as a function of the input size

| **Algorithm** ***arrayMax***(***A***, ***n***) | # operations |
|---|---|
| *currentMax* ← *A*[0] | 2 |
| **for** (*i* =1; i<n; i++) | 2***n*** |
| (i=1 once, i<n n times, i++ (n-1) times:post increment) | |
| **if** *A*[*i*] > *currentMax* **then** | 2(***n*** – 1) |
| *currentMax* ← *A*[*i*] | 2(***n*** – 1) |
| **return** *currentMax* | 1 |
| Total | 6***n*** –1 |

# Estimating Running Time



- Algorithm ***arrayMax*** executes $6n - 1$ primitive operations in the worst case.

  Define:

  $a$ = Time taken by the fastest primitive operation

  $b$ = Time taken by the slowest primitive operation

- Let $T(n)$ be worst-case time of ***arrayMax***. Then

$$a\,(6n - 1) \leq T(n) \leq b(6n - 1\,)$$

- Hence, the running time $T(n)$ is bounded by two linear functions

# Growth Rate of Running Time

- Changing the hardware/ software environment
  - Affects $T(n)$ by a constant factor, but
  - Does not alter the growth rate of $T(n)$
- The linear growth rate of the running time $T(n)$ is an intrinsic/basic property of algorithm ***arrayMax***



# Function of Growth rate

| Function | Name |
|---|---|
| $c$ | Constant |
| $\log N$ | Logarithmic |
| $\log^2 N$ | Log-squared |
| $N$ | Linear |
| $N \log N$ | $N$ log $N$ |
| $N^2$ | Quadratic |
| $N^3$ | Cubic |
| $2^N$ | Exponential |

Functions in order of increasing growth rate

# Big-Oh Notation

- To simplify the running time estimation, for a function *f(n)*, we ignore the constants and lower order terms.

Example: $10n^3+4n^2-4n+5$ is $O(n^3)$.